

Estimaciones de SW

Video: <https://youtu.be/TEiOcgjv8jk?si=WlxA6y61Eot23mtT>

El proceso de estimación entrega algo cuantitativo, un número.

¿Cuándo estimamos en el proceso de desarrollo de software? Nos exigen hacer estimaciones al principio, pero es donde menos información tenemos, entonces las estimaciones no son buenas. Conforme vamos avanzando, adquiriendo más información, y es aquí donde podemos equivocarnos menos en las estimaciones.

Tienen un valor de **probabilidad** asociado. Te puede salir bien o no. Informo un valor cualitativo asociado a una probabilidad. Pero se suelen asumir como promesas.

Pasan dos escenarios:

- Se estima de menos por miedo de que el cliente rechace la propuesta
- Se sobreestima

La estimación es la segunda cosa más complicada, la primera es la licitación.

Hay que saber que a medida que vaya cambiando el alcance y las necesidades del producto, se debe considerar la RE estimación. Si los requerimientos cambian, las estimaciones también.

Luego cuando uno planifica lo hace en base a una estimación hecha previamente. Pero no es lo mismo.

Las estimaciones no son precisas, tienen un error asociado, hay que tratar que ese error sea el menor posible. Típico error => Se consideran solo las horas de programación KE, la programación no debería llevar más del 35% del esfuerzo total de un proyecto. Hay que considerar otras actividades como testing, hablar con el cliente, la arquitectura, etc.

Otro problema es de quien estima, se usa una técnica "Juicio del experto", pero quien estima no es quien va a hacer el trabajo y se hace una mala estimación.

Técnicas fundamentales de estimación

Contar: pero como cuentas algo intangible? Hace muchos años se contaban las líneas de código. Problemas por todos lados, las líneas de código dependen del lenguaje, del programador, no tienen relación con la complejidad del problema, más líneas de código no conlleva más calidad, imposible de estimar a un comienzo.

Métodos utilizados

- Basados en la experiencia
 - Datos históricos: La experiencia de otros proyectos de otra gente me puede servir a mí. Se arman un conjunto de parámetros (Tecnología, Dominio del problema, Cantidad de personas, Tiempo que llevo, etc) y se comienza a generar una base de datos, y se puede hacer una proyección. Los enfoques ágiles van en contra de esto, porque dicen que la experiencia la puede capitalizar la misma gente en ese proyecto, pero no se puede extrapolar la

experiencia como equipo a otro proyecto o a alguien más. Además de que depende del contexto en el que vivimos y de circunstancias externas a nosotros.

- Juicio experto:
 - Puro: El más usado (75% de organizaciones lo usan). Hay una dependencia muy fuerte con esta persona. Estima desde lo que él considera que es complejo y que no, desde lo que él considera para estimar. Cuando esto se traslada a quien realmente va a hacer el trabajo, hay muchas desviaciones.
 - Delphi: Es grupal. Se toma como base para el Poker planning. cada uno hace una estimación individual y luego la comparte con los demás, para que converja a una sola estimación.
 - Analogía: De todos los datos históricos registrados, me fijo solo los que son similares al proyecto que se quiere hacer. Abstraigo de todos los datos históricos un subgrupo.
- Basados en los recursos
 - Basado en el mercado
 - Basados en los componentes del producto o en el proceso de desarrollo
 - Métodos algorítmicos

*Poker planning o estimation: User canonica.

Estimaciones ágiles

Fuerte propuesta de que estime el que hace el trabajo.

A diferencia del enfoque tradicional en el que se estimaba en este orden: Tamaño, Esfuerzo, Calendario, Costo, Recursos Críticos.

En el enfoque ágil también se analiza el Tamaño (US, Épicas o Temas). Para las user se usan Story Points, que es el peso de la user. Es una unidad homogeneizadora que me permite comparar entre users.

Las estimaciones ágiles son relativas, por comparación.

3 dimensiones/variables que conforman el **Story Point**:

- Complejidad
- Esfuerzo: estimado por el que hace el trabajo
- Duda: Cuanta información nos falta para ser más adecuados

Es la forma de justificar ese peso de la user story.

Gran problemática => El costo más grande son las horas Esfuerzo

Se suele confundir **Esfuerzo** con **tamaño**:

- El tamaño es más objetivo, es la complejidad que tiene hacer eso
- El esfuerzo es más personal, es la cantidad de horas que necesita esa persona para hacerlo

Y **Esfuerzo** con **calendario**

- El esfuerzo son horas ideales, solo se cuentan horas de trabajo productivo

- En el calendario en cambio es decir por ej. te lo entrego el viernes, pero si tu jefe te da más cosas para hacer, entonces puede que no llegues a lo que te comprometiste, aunque hayas trabajado un montón de horas. No está bueno estimar solo en calendario, sino que sea una derivación del esfuerzo.

Escalas para estimar el tamaño

- Tamaño por números
- Talles de remeras
- Serie 2^n
- Fibonacci

Una vez elegida, no cambiarla, perdemos estimaciones hechas

Velocidad

Se calcula sumando los Story Points de las US que le presenté al cliente y el cliente me aceptó. Es cuanto producto le entregué al cliente después de una iteración de trabajo. Es una métrica que no se estima, sino que se calcula al final de la iteración.

Se compara con la suma de puntos a los que te habías comprometido. Analizar las razones, aprendes para la próxima. Se pueden trabajar con rangos de puntos para tener un margen.

*Con el product owner te comunicas en términos de Story Points. Con el cliente real no, le decís un tiempo ej. en un año este volumen de requerimientos hechos, eso puede variar en términos de qué funcionalidades vas a hacer pero no en términos de cantidad de puntos o volumen (cantidad) vas a hacer.

¿Cómo se estima un proyecto?

No se estima, sino que se deriva tomando el número total de story points de sus US y dividiéndolo por la velocidad del equipo.

Priorizamos las US, se estima el tamaño del proyecto en story points, se deriva la duración según la velocidad del equipo.

Poker Estimation: Es la técnica con cartas para estimar.

¿Cómo interpretamos las estimaciones?

Cuando uno trabaja con iteraciones cortas, tiene que trabajar con piezas lo más pequeñas posibles. No se trabaja con porcentajes de avance, sino con una gestión binaria (está terminado o no). En INVEST, la ese es Small, pequeño quiere decir que entraba en una iteración. Ej. una user de 20 puntos no la puedes terminar en un sprint, salvo que sea una user que no puedes partir por alguna razón de cohesión o algo, pero todos se comprometen y están al tanto de la complejidad que se asume y una velocidad anormal.

Es recomendable para entrar a un sprint los valores de 1, 2,3 y 5. El equipo debe trabajar en partir stories para que las que se trabajen sean de este tamaño.

Cómo hacer que la estimación sea útil? Estimar pero no aferrarse a las estimaciones. Estar dispuestos a revisarlas cada vez que se considere necesario. Aprender de eso.

